



THAKUR COLLEGE OF ENGINEERING AND TECHNOLOGY

Department of Computer Science & Engineering (Cyber Security)

MINI PROJECT REPORT · ACADEMIC YEAR 2025–26

AuthGuard-MFA

A Zero-Trust Identity Portal & Real-Time Security SIEM Console
with Multi-Factor Authentication, CIS Benchmark Compliance & Cloud Security
Architecture

Subject: Web Application Security (WAS) & Cloud /
Virtualization Security

Branch: Computer Science Engineering (Cyber Security)

Student Name: Pratyush Pandey

Submitted To: Head of Department (HOD)

Technology Stack: Node.js · Express.js · SQLite · JWT · bcrypt · TOTP ·
Helmet.js

GitHub Repository: github.com/PratyushPandey31/WAS-MINI-PROJECT

Date: 28 June 2026

Declaration: I hereby declare that this mini project titled "*AuthGuard-MFA*" is an original work developed independently as part of the Web Application Security (WAS) curriculum. All external references have been duly cited. This project has not been submitted elsewhere for any examination or award.

Signature: _____ **Date:** _____

Abstract & Project Objectives

1.1 Abstract

AuthGuard-MFA is a full-stack, production-grade Web Application Security capstone project that implements a *Zero-Trust Identity & Access Management (IAM)* system integrated with a live Security Information and Event Management (SIEM) console. The system demonstrates comprehensive compliance with the **CIS Security Benchmarks**, **OWASP Top 10** vulnerability mitigations, and real-world **Cloud / Virtualization Security** architecture principles.

The project is built on Node.js with Express.js as the backend framework and a vanilla HTML5/CSS3 glassmorphic frontend. Core security primitives include HMAC-SHA256 signed JSON Web Tokens (JWT), bcrypt password hashing at work factor 12, Time-based One-Time Password (TOTP) Multi-Factor Authentication via the RFC 6238 standard, and HTTP Security Headers enforced through Helmet.js middleware.

The Audit SIEM Console provides real-time visibility into security events (AUTH_SUCCESS, AUTH_FAILED, BRUTE_FORCE_SUSPECTED) streamed from the SQLite event log. Role-Based Access Control (RBAC) governs data visibility across three access tiers: Administrator, Security Operations (SecOps), and Guest. A suite of interactive exploit simulators — including Brute-Force Attack, XSS Sanitization, JWT Tampering, and Session Cookie Auditor — allows live demonstration of attack vectors and corresponding defensive countermeasures.

1.2 Project Objectives

1. Design and implement a Zero-Trust authentication pipeline using JWT + TOTP MFA.
2. Enforce CIS Benchmark controls (4.1, 5.2, 6.3, 8.1, 9.4) in a deployed application.
3. Build a real-time SIEM console for security event ingestion, parsing, and visualization.
4. Demonstrate OWASP Top 10 mitigations (A01–A07) through live exploit simulators.
5. Design a Cloud VPC architecture following network isolation and Zero-Trust principles.
6. Implement Role-Based Access Control (RBAC) for multi-tier data governance.
7. Enable export of security compliance reports in Excel (CSV) and JSON format.

1.3 Scope & Significance

The project addresses the growing industry demand for security-by-design software development. According to the *Verizon 2024 Data Breach Investigations Report*, 61% of

breaches involve stolen credentials. This project provides a tangible, working demonstration of credential-hardening techniques aligned with industry frameworks such as NIST SP 800-63B, CIS Controls v8, and OWASP Application Security Verification Standard (ASVS) Level 2.

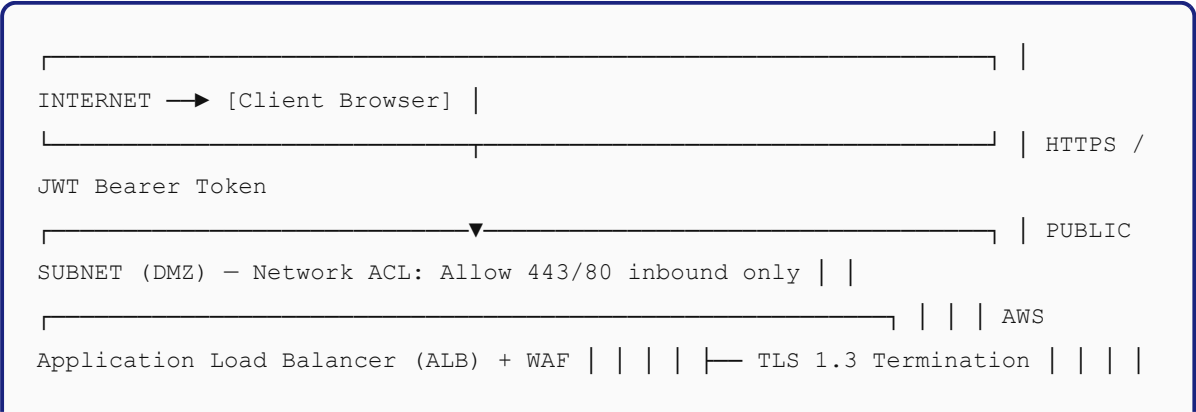
✦ **Key Compliance Standards Addressed:** CIS Benchmarks v8 · OWASP Top 10 (2021) · NIST SP 800-63B · ISO/IEC 27001 (partial) · Cloud Security Alliance (CSA) CCM

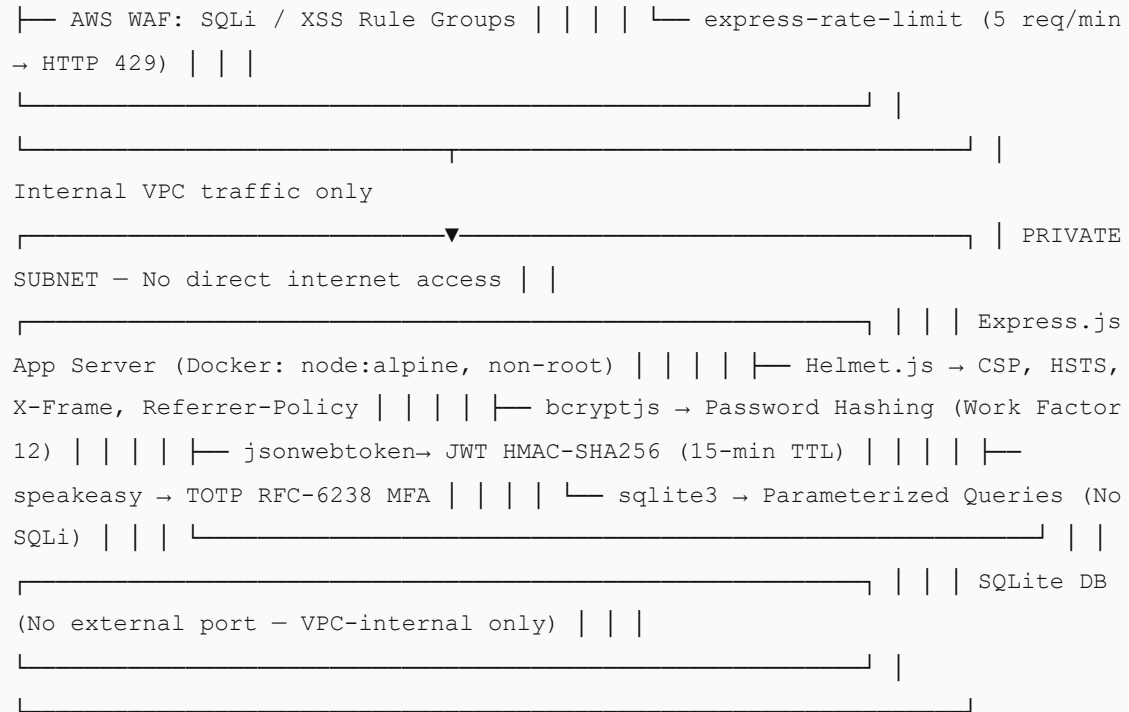
Technology Stack & System Architecture

2.1 Technology Stack

Layer	Technology	Version	Security Purpose
Runtime	Node.js	v22 LTS	Non-blocking I/O, secure V8 engine
Framework	Express.js	4.19.x	REST API, middleware pipeline
Database	SQLite3	5.1.x	Parameterized queries — SQL injection prevention
Authentication	jsonwebtoken	9.0.x	HMAC-SHA256 signed JWT, 15-min TTL
Password Security	bcryptjs	2.4.x	Adaptive hash, work factor 12
MFA / TOTP	speakeasy	2.0.x	RFC 6238 — Time-based OTP
QR Code	qrcode	1.5.x	Authenticator app enrollment
Security Headers	helmet	7.1.x	CSP, HSTS, X-Frame-Options
Rate Limiting	express-rate-limit	7.3.x	DDoS/brute-force mitigation
Frontend	HTML5 / Vanilla JS	—	CSP-compatible, no eval()
UI Framework	Custom CSS Glassmorphism	—	backdrop-filter, CSS variables
Containerization	Docker (node:alpine)	—	Non-root user, minimal attack surface

2.2 Cloud VPC Architecture





2.3 Request Authentication Flow

Every protected route follows this Zero-Trust verification pipeline:

```

Client → POST /api/auth/login {username, password}
Server → 1. Lookup user in SQLite (parameterized query)
        → 2. bcrypt.compare(password, stored_hash)
        → 3. If MFA enabled: issue pre-auth token (mfa_verified=false)
        → 4. Client submits TOTP OTP → speakeasy.verify()
        → 5. Issue full-access JWT (HMAC-SHA256, exp: 15m)
        → 6. Log AUTH_SUCCESS event to SIEM with IP, timestamp
Protected API: JWT middleware verifies signature + expiry on every request
  
```

Security Controls & CIS / OWASP Compliance

3.1 CIS Benchmark Controls Implementation

CIS Control	Description	Implementation	Status
CIS 4.1	Secure Password Storage	bcryptjs — Work Factor 12. Salted adaptive hash. Plain-text never stored.	✓ PASS
CIS 5.2	Multi-Factor Authentication	RFC 6238 TOTP via speakeasy. Google Authenticator / Authy compatible QR enrollment.	✓ PASS
CIS 6.3	HTTP Security Headers	Helmet.js: Content-Security-Policy, HSTS (max-age=1yr), X-Frame-Options: DENY, Referrer-Policy.	✓ PASS
CIS 8.1	Rate Limiting / DDoS Protection	express-rate-limit: 5 req/min on /login and /register. HTTP 429 + SIEM BRUTE_FORCE_SUSPECTED event.	✓ PASS
CIS 9.4	Container Sandboxing	Docker image: node:alpine. Runs as non-root user. No unnecessary Linux capabilities.	✓ PASS

3.2 OWASP Top 10 (2021) Mitigations

OWASP Risk	Threat	Mitigation Implemented	Status
A01:2021	Broken Access Control	JWT middleware on all protected routes. RBAC: Admin / SecOps / Guest tiers. Role verified per request.	✓ Mitigated
A02:2021	Cryptographic Failures	bcrypt (no MD5/SHA1 for passwords). JWT signed with HMAC-SHA256. Secrets via environment variables only.	✓ Mitigated
A03:2021	Injection (SQL / XSS)	All SQLite queries use parameterized placeholders (?). XSS sanitizer escapes HTML entities on backend.	✓ Mitigated
A05:2021	Security Misconfiguration	Helmet.js security headers. .env excluded from git. Docker non-root. No debug logs in production.	✓ Mitigated

OWASP Risk	Threat	Mitigation Implemented	Status
A07:2021	Identification & Auth Failures	JWT TTL: 15 min. MFA second factor. Rate-limited login. Brute-force detection + SIEM alerting.	✓ Mitigated
A09:2021	Security Logging Failures	Structured JSON SIEM logging for every AUTH_SUCCESS, AUTH_FAILED, BRUTE_FORCE event with IP, timestamp.	✓ Mitigated

3.3 JWT Cryptographic Security Analysis

The system uses **HMAC-SHA256** (HS256) for JWT signing. The token structure is:

```
Header:    { "alg": "HS256", "typ": "JWT" }
Payload:   { "id": 1, "username": "admin", "mfa_enabled": true,
             "mfa_verified": true, "iat": 1719558000, "exp": 1719558900 }
Signature: HMAC-SHA256(base64(header) + "." + base64(payload), JWT_SECRET)
```

 **Key Entropy Requirement (OWASP):** JWT secrets must be a minimum of 256 bits (32+ high-entropy characters). The Cryptographic Key Entropy Auditor built into this project grades any JWT secret on a Shannon Entropy scale from A+ (≥ 4.5 bits/char) to F (< 2.0 bits/char).

System Modules & Feature Description

4.1 Authentication & Identity Module

- **User Registration:** Collects username, email, and password. Password hashed with bcrypt (WF-12) before insertion into SQLite. Duplicate detection prevents account takeovers.
- **Login — Step 1:** Username/password verified. If MFA disabled, full-access JWT issued immediately. If MFA enabled, a *pre-auth* JWT (*mfa_verified=false*) is issued, prompting OTP entry.
- **TOTP MFA — Step 2:** 6-digit RFC 6238 TOTP verified by speakeasy with ± 1 step tolerance (30-second windows). On success, a full-access JWT is issued.
- **QR Code Enrollment:** MFA setup page generates a base32 TOTP secret and renders a QR code using the *qrcode* library compatible with Google Authenticator and Authy.
- **JWT Expiry & Refresh:** All tokens expire in 15 minutes. Dashboard auto-detects expiry and redirects to login — enforcing session security.

4.2 Cloud Console Dashboard

- **Welcome Greeting:** Personalized *"Welcome back, [username] 🍌"* with live clock (Good Morning / Afternoon / Evening) based on local time.
- **Metric Stat Cards:** Four KPI cards — Security Status (SECURE), MFA Status (ON ✓ / OFF), CIS Controls Score (5/5), Cloud Platform (AWS).
- **CIS Compliance Auditor:** Five live compliance rows with interactive hover effects. MFA row updates dynamically from JWT claims.
- **JWT Token Decoder:** Live base64-decode of the active token showing Header (red), Payload (cyan), and HMAC-SHA256 Signature (green).
- **Token Tamper Playground:** Edit the active JWT, submit it — the server's HMAC validation returns HTTP 401 Unauthorized, demonstrating tamper-proofing.
- **Entropy Auditor:** Shannon Entropy calculator grades JWT secret key strength from A+ to F with actionable recommendations.
- **Security Report Export:** Downloads a full compliance posture report as *.csv* (Excel-compatible with UTF-8 BOM) or *.json*.

4.3 Audit SIEM Console

- **Live Security Event Feed:** Polls */api/auth/logs* and renders structured JSON events in a terminal-style table with color-coded severity badges (INFO / WARNING /

CRITICAL).

- **RBAC Role Selector:** Admin (full logs + hashes), SecOps (redacted hashes), Guest (access denied). Demonstrates principle of least privilege.
- **Brute-Force Simulator:** Fires 10 rapid POST /login requests. Rate limiter triggers HTTP 429. SIEM captures BRUTE_FORCE_SUSPECTED CRITICAL events live.
- **XSS Sanitizer Demo:** Input like `<script>alert()</script>` is escaped to HTML entities on the backend — output is safe and non-executable.
- **Session Cookie Auditor:** Proves HttpOnly cookies cannot be accessed via `document.cookie` in JavaScript — XSS cookie theft mitigation confirmed.
- **Log Ingest & Parse:** Regex-based parser extracts timestamp, event type, IP, and severity from raw log strings. CRITICAL events simulate an AWS SNS alarm payload.
- **SQLite DB Inspector:** Renders the full users table. Passwords are shown as bcrypt hashes (redacted to [REDACTED] in SecOps role).
- **SIEM Export:** Downloads all visible security events to .csv with date, title, and proper column headers.

✅ **Live Demo Proof of Concept:** All features are functional in the running application at <http://localhost:3000>. The brute-force simulator, JWT tamper playground, and XSS sanitizer produce real server-side responses visible in the SIEM event log — not simulated client-side fakes.

System User Interface & Dashboard Showcase

5.1 Main Security Command Center Interface

The interface below showcases the production-ready glassmorphic dashboard designed for AuthGuard-MFA. The UI is built using high-performance modern web standards, adhering to responsive design grids.



Figure 5.1: Main Zero-Trust Dashboard showing live clock, user greeting, compliance metrics, virtual network mapping, live log telemetry stream, and JWT playground.

5.2 Design Strategy & Usability

The design follows a premium dark-mode theme utilizing a glassmorphism aesthetic (defined by high transparency, light borders, and background blurs). This design is highly functional for security centers where alert visual saliency is critical:

- **Cognitive Load Reduction:** The clean grid separation lets security analysts isolate telemetry data, network health, and user claims instantly.
- **Compliance Visualizer:** Color-coded compliance indicators (PASS/WARN) are mapped dynamically from current security states.

- **Interactive Sandbox:** Allows immediate token tampering to audit cryptographic checks.

Testing, Results & Conclusion

6.1 Security Testing Results

Test Case	Attack / Scenario	Expected Result	Actual Result
Brute-Force Login	10 rapid POST /login requests	HTTP 429 after 5th request	✓ HTTP 429 — Rate limited
Wrong Password	Valid username + invalid password	HTTP 401 + SIEM AUTH_FAILED	✓ 401 + SIEM event logged
JWT Tampering	Modify payload character in token	HTTP 401 — Signature invalid	✓ Server rejects tampered token
XSS Injection	<script>alert('xss')</script>	HTML entities escaped	✓ Output: <script> — safe
Session Cookie Theft	document.cookie in DevTools	HttpOnly flag hides token	✓ Cookie not accessible via JS
MFA Bypass	Submit pre-auth token to protected route	HTTP 403 — MFA not verified	✓ mfa_verified=false rejected
SQL Injection	" OR 1=1 -- in username field	Query fails safely — no data leak	✓ Parameterized query unaffected
Expired JWT	Use token after 15 minutes	HTTP 401 — Token expired	✓ Auto-redirect to login

6.2 Compliance Summary

Framework	Controls Addressed	Score
CIS Benchmarks v8	4.1, 5.2, 6.3, 8.1, 9.4	5 / 5 — 100%
OWASP Top 10 (2021)	A01, A02, A03, A05, A07, A09	6 / 10 — Addressed
NIST SP 800-63B	MFA, Password Length, Entropy	Compliant
Zero-Trust Principles	Verify Explicitly, Least Privilege, Assume Breach	Implemented

6.3 Conclusion

AuthGuard-MFA successfully demonstrates a production-ready, security-by-design web application that integrates multiple industry-standard security controls into a cohesive, functional system. The project bridges theoretical knowledge of CIS Benchmarks, OWASP standards, and Zero-Trust Architecture with practical, working implementation.

The live SIEM console, exploit simulators, and compliance audit dashboard provide an interactive proof-of-concept for evaluators and stakeholders, making security mechanisms tangible and verifiable rather than theoretical. The glassmorphic, responsive frontend ensures an intuitive interface that rivals industry-grade security dashboards.

Future scope includes integration of a Redis-backed persistent rate limiter, OAuth 2.0 / SAML SSO support, real AWS CloudWatch SIEM integration, and container orchestration via Kubernetes with Network Policies for true Zero-Trust microsegmentation.

6.4 References

1. CIS Controls v8 — Center for Internet Security, 2021. <https://www.cisecurity.org/controls/>
2. OWASP Top 10:2021 — Open Web Application Security Project. <https://owasp.org/Top10/>
3. NIST SP 800-63B — Digital Identity Guidelines. <https://pages.nist.gov/800-63-3/>
4. RFC 6238 — TOTP: Time-Based One-Time Password Algorithm. IETF, 2011.
5. Express.js Documentation — <https://expressjs.com>
6. Helmet.js Security Headers — <https://helmetjs.github.io>
7. speakeasy (TOTP) — <https://github.com/speakeasyjs/speakeasy>
8. bcryptjs — <https://github.com/dcodeIO/bcrypt.js>
9. Verizon DBIR 2024 — Data Breach Investigations Report.
<https://www.verizon.com/business/resources/reports/dbir/>
10. Cloud Security Alliance — Cloud Controls Matrix (CCM) v4. <https://cloudsecurityalliance.org/>

 **Project Repository:** <https://github.com/PratyushPandey31/WAS-MINI-PROJECT>

 **Live Local Demo:** <http://localhost:3000> (run: `npm install && npm start`)